

LAPORAN PROYEK
EVALUASI AKHIR SEMESTER (EAS)
PEMROGRAMAN BASIS DATA
IMPLEMENTASI SISTEM BASIS DATA RESTORAN MENGGUNAKAN MYSQL



Disusun Oleh :

Kelompok 9

Farida Nur Intan	IK2411013
Gefran	IK2411029
Tiara Nuriani	IK2411024
Magfakhrani Nur Fauzia Armin	IK2411050

Program Studi Informatika

Fakultas Ilmu Komputer

Universitas Mega Buana Palopo

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah memberikan dampak yang sangat besar terhadap berbagai bidang usaha, salah satunya adalah bidang kuliner. Restoran merupakan salah satu jenis usaha yang setiap hari melakukan banyak aktivitas pengelolaan data, seperti pencatatan data pelanggan, pengelolaan menu makanan dan minuman, pencatatan transaksi pemesanan, serta penyusunan laporan penjualan. Apabila seluruh aktivitas tersebut masih dilakukan secara manual, maka akan menimbulkan berbagai permasalahan seperti kesalahan pencatatan, kehilangan data, kesulitan dalam pencarian informasi, serta proses pembuatan laporan yang membutuhkan waktu cukup lama.

Basis data merupakan komponen penting dalam sebuah sistem informasi karena berfungsi sebagai tempat penyimpanan data yang terstruktur sehingga data dapat dikelola secara efektif, efisien, dan aman. Dengan menggunakan sistem basis data, proses penyimpanan, pencarian, pengolahan, serta pelaporan data dapat dilakukan secara otomatis sehingga mampu meningkatkan kualitas pelayanan kepada pelanggan.

Pada proyek Evaluasi Akhir Semester (EAS) mata kuliah Pemrograman Basis Data ini, dibangun sebuah Sistem Basis Data Restoran menggunakan MySQL. Sistem ini dirancang untuk mengelola data pelanggan, kategori menu, menu restoran, transaksi pemesanan, detail pesanan, serta audit log aktivitas sistem. Selain menggunakan tabel dan relasi antar tabel, sistem juga mengimplementasikan berbagai fitur lanjutan yang tersedia pada MySQL, seperti Stored Procedure, Function, Trigger, Cursor, Transaction Control, Exception Handling, dan Indexing. Implementasi fitur-fitur tersebut bertujuan untuk meningkatkan efisiensi proses pengolahan data, menjaga konsistensi data, mempercepat proses pencarian informasi, serta mempermudah proses pembuatan laporan.

Dengan adanya sistem basis data ini diharapkan seluruh aktivitas operasional restoran dapat dilakukan secara lebih cepat, akurat, dan terorganisir, sehingga mampu mendukung proses pengambilan keputusan berdasarkan data yang tersimpan di dalam database.

1.2 Tujuan Proyek

Tujuan dari pelaksanaan proyek Sistem Basis Data Restoran ini adalah sebagai berikut.

1. Merancang dan membangun sistem basis data restoran menggunakan MySQL.
2. Membuat struktur tabel yang saling berelasi menggunakan Primary Key dan Foreign Key.
3. Mengimplementasikan proses pengolahan data menggunakan Stored Procedure.
4. Mengimplementasikan Function untuk membantu proses perhitungan subtotal dan diskon.
5. Mengimplementasikan Trigger untuk melakukan otomatisasi pengurangan stok serta pencatatan audit log.
6. Mengimplementasikan Cursor untuk menghasilkan laporan rekap penjualan bulanan.
7. Mengimplementasikan Transaction Control menggunakan START TRANSACTION, SAVEPOINT, COMMIT, dan ROLLBACK.
8. Mengimplementasikan Exception Handling untuk menangani kesalahan saat proses transaksi.
9. Mengimplementasikan Indexing guna meningkatkan performa pencarian data.
10. Melakukan pengujian terhadap seluruh objek database yang telah dibuat.

1.3 Ruang Lingkup Sistem

Agar pembahasan tidak terlalu luas, maka ruang lingkup proyek ini dibatasi pada beberapa aspek berikut.

1. Sistem dibangun menggunakan DBMS MySQL.
2. Database terdiri dari enam tabel utama yaitu tabel pelanggan, kategori_menu, menu, pesanan, detail_pesanan, dan audit_log.
3. Sistem hanya berfokus pada pengelolaan data restoran dan belum mencakup antarmuka berbasis web maupun desktop.

4. Sistem menyediakan pengelolaan data pelanggan, menu makanan, transaksi pemesanan, detail pesanan, serta laporan penjualan.
5. Sistem mengimplementasikan Stored Procedure, Function, Trigger, Cursor, Transaction Control, Exception Handling, dan Indexing sebagai bagian dari fitur lanjutan MySQL.
6. Laporan yang dihasilkan meliputi laporan penjualan harian dan rekap penjualan bulanan.
7. Seluruh pengujian dilakukan menggunakan MySQL melalui phpMyAdmin atau MySQL Workbench.

BAB II

ANALISIS DAN PERANCANGAN

2.1 Deskripsi Kebutuhan Sistem

1. Gambaran Umum Sistem

Sistem Basis Data Restoran merupakan sebuah sistem yang dirancang untuk membantu proses pengelolaan data operasional restoran secara terstruktur. Sistem ini dibangun menggunakan MySQL sebagai Database Management System (DBMS) dengan tujuan untuk menyimpan, mengelola, dan mengolah data transaksi restoran secara efektif dan efisien.

Dalam kegiatan operasional restoran terdapat berbagai aktivitas yang dilakukan setiap hari, seperti pencatatan data pelanggan, pengelolaan kategori menu, pengelolaan menu makanan dan minuman, pencatatan transaksi pemesanan, penyimpanan detail pesanan, hingga pembuatan laporan penjualan. Apabila seluruh aktivitas tersebut dilakukan secara manual, maka berpotensi menimbulkan kesalahan pencatatan, kehilangan data, serta memperlambat proses pelayanan kepada pelanggan.

Untuk mengatasi permasalahan tersebut, dibangun sebuah sistem basis data yang mampu mengelola seluruh data secara terintegrasi. Sistem ini memanfaatkan berbagai fitur lanjutan MySQL seperti Stored Procedure, Function, Trigger, Cursor, Transaction Control, Exception Handling, dan Indexing sehingga proses pengolahan data dapat dilakukan secara otomatis dan lebih aman. Struktur database terdiri atas enam tabel utama, yaitu pelanggan, kategori_menu, menu, pesanan, detail_pesanan, dan audit_log. Selain itu, sistem menyediakan prosedur untuk pembuatan pesanan, perhitungan subtotal dan diskon, laporan penjualan harian, serta rekap penjualan bulanan.

2. Kebutuhan Fungsional

Kebutuhan fungsional merupakan fungsi-fungsi yang harus dapat dilakukan oleh sistem agar seluruh proses operasional restoran dapat berjalan dengan baik.

Adapun kebutuhan fungsional sistem adalah sebagai berikut :

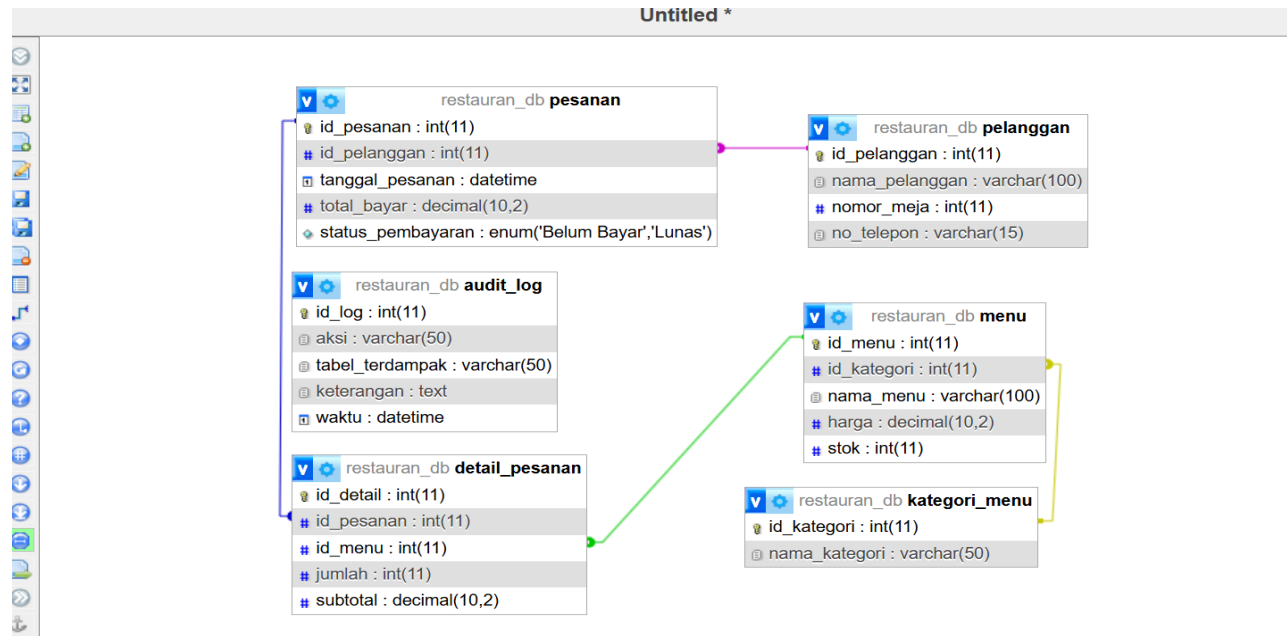
- Sistem mampu menyimpan data pelanggan restoran.
- Sistem mampu menyimpan data kategori menu.
- Sistem mampu menyimpan data menu makanan dan minuman beserta harga dan stok.
- Sistem mampu mencatat transaksi pemesanan pelanggan.
- Sistem mampu mencatat detail setiap pesanan.
- Sistem mampu menghitung subtotal pesanan secara otomatis menggunakan Function.
- Sistem mampu menghitung potongan diskon berdasarkan total pembayaran menggunakan Function.
- Sistem mampu mengurangi stok menu secara otomatis setelah transaksi menggunakan Trigger.
- Sistem mampu mencatat perubahan status pembayaran ke dalam tabel audit log.
- Sistem mampu menghasilkan laporan penjualan harian.
- Sistem mampu menghasilkan laporan rekap penjualan bulanan menggunakan Cursor.
- Sistem mampu menangani kesalahan input data melalui Exception Handling.
- Sistem mampu menjaga konsistensi data menggunakan Transaction Control.
- Sistem mampu mempercepat proses pencarian data menggunakan Index.

3. Kebutuhan Non Fungsional

- Perangkat Lunak
- Sistem Operasi Windows 10 atau Windows 11.
- XAMPP sebagai web server lokal.
- phpMyAdmin untuk pengelolaan database.
- MySQL/MariaDB sebagai DBMS.
- Visual Studio Code sebagai editor.
- Perangkat Keras
- Processor minimal Intel Core i3 atau AMD Ryzen 3.
- RAM minimal 4 GB.
- Ruang penyimpanan minimal 20 GB.
- Monitor dengan resolusi minimal 1366×768 piksel.

2.2 Entity Relationship Diagram (ERD)

Entity Relationship Diagram (ERD) merupakan rancangan hubungan antar entitas yang digunakan sebagai dasar pembangunan database. Pada sistem ini terdapat enam entitas utama, yaitu pelanggan, kategori_menu, menu, pesanan, detail_pesanan, dan audit_log. Hubungan antar entitas dirancang menggunakan konsep one-to-many (1:N), sehingga mampu menggambarkan keterkaitan data secara jelas dan menghindari redundansi data.



Penjelasan ERD

- Satu pelanggan dapat memiliki banyak pesanan.
- Satu kategori menu dapat memiliki banyak menu.
- Satu pesanan dapat memiliki banyak detail pesanan.
- Satu menu dapat muncul pada banyak detail pesanan.
- Audit log digunakan untuk menyimpan aktivitas sistem yang dihasilkan oleh Trigger.

2.3 Relasi Antar Tabel

Hubungan antar tabel pada sistem basis data restoran dibangun menggunakan Primary Key dan Foreign Key untuk menjaga integritas data.

1. Relasi Tabel pelanggan dengan pesanan

Tabel pelanggan memiliki relasi one-to-many dengan tabel pesanan. Setiap pelanggan dapat melakukan lebih dari satu transaksi pemesanan, sedangkan satu transaksi hanya dimiliki oleh satu pelanggan. Relasi ini menggunakan atribut `id_pelanggan` sebagai foreign key pada tabel pesanan.

2. Relasi Tabel kategori_menu dengan menu

Tabel kategori_menu memiliki relasi one-to-many dengan tabel menu. Setiap kategori dapat memiliki banyak menu, sedangkan satu menu hanya termasuk dalam satu kategori. Hubungan tersebut menggunakan atribut `id_kategori` sebagai foreign key pada tabel menu.

3. Relasi Tabel pesanan dengan detail_pesanan

Tabel pesanan memiliki relasi one-to-many dengan tabel detail_pesanan. Satu transaksi dapat terdiri dari beberapa item menu yang berbeda. Hubungan ini menggunakan atribut `id_pesanan` sebagai foreign key pada tabel detail_pesanan.

4. Relasi Tabel menu dengan detail_pesanan

Tabel menu memiliki relasi one-to-many dengan tabel detail_pesanan. Satu jenis menu dapat muncul pada banyak transaksi pemesanan. Relasi ini menggunakan atribut `id_menu` sebagai foreign key pada tabel detail_pesanan.

5. Relasi Audit Log

Tabel audit_log tidak memiliki foreign key karena berfungsi sebagai tabel pencatat aktivitas sistem. Data pada tabel ini dihasilkan secara otomatis oleh Trigger ketika terjadi perubahan tertentu pada database.

2.4 Struktur Tabel

1. Tabel pelanggan

Field	Tipe Data	Keterangan
id_pelanggan	INT	Primary Key
nama_pelanggan	VARCHAR(100)	Nama pelanggan
nomor_meja	INT	Nomor meja
no_telepon	VARCHAR(15)	Nomor telepon

Penjelasan: Tabel ini menyimpan identitas pelanggan yang melakukan transaksi di restoran. Setiap pelanggan memiliki id_pelanggan sebagai identitas unik.

2. Tabel kategori_menu

Field	Tipe Data	Keterangan
id_kategori	INT	Primary Key
nama_kategori	VARCHAR(50)	Nama kategori menu

Penjelasan: Tabel ini digunakan untuk mengelompokkan menu ke dalam beberapa kategori, seperti makanan utama, minuman, camilan, dessert, dan paket hemat.

3. Tabel menu

Field	Tipe Data	Keterangan
id_menu	INT	Primary Key
id_kategori	INT	Foreign Key
nama_menu	VARCHAR(100)	Nama menu
harga	DECIMAL(10,2)	Harga menu
stok	INT	Jumlah stok

Penjelasan: Tabel ini menyimpan data menu restoran beserta harga dan stok. Setiap menu dikaitkan dengan satu kategori menggunakan id_kategori.

4. Tabel pesanan

Field	Tipe Data	Keterangan
id_pesanan	INT	Primary Key
id_pelanggan	INT	Foreign Key
tanggal_pesanan	DATETIME	Tanggal transaksi
total_bayar	DECIMAL(10,2)	Total pembayaran
status_pembayaran	ENUM	Status pembayaran

Penjelasan: Tabel ini menyimpan data transaksi yang dilakukan pelanggan. Status pembayaran digunakan untuk membedakan transaksi yang sudah lunas dan yang belum lunas.

5. Tabel detail_pesanan

Field	Tipe Data	Keterangan
id_detail	INT	Primary Key
id_pesanan	INT	Foreign Key
id_menu	INT	Foreign Key
jumlah	INT	Jumlah menu
subtotal	DECIMAL(10,2)	Nilai subtotal

Penjelasan: Tabel ini menyimpan rincian menu yang dipesan pada setiap transaksi, termasuk jumlah menu dan nilai subtotal.

6. Tabel audit_log

Field	Tipe Data	Keterangan
id_log	INT	Primary Key
aksi	VARCHAR(50)	Jenis aktivitas
tabel_terdampak	VARCHAR(50)	Nama tabel
keterangan	TEXT	Informasi aktivitas
waktu	DATETIME	Waktu kejadian

Penjelasan: Tabel audit log digunakan untuk mencatat aktivitas penting yang dilakukan oleh Trigger, seperti perubahan status pembayaran dan penghapusan data menu.

BAB III

IMPLEMENTASI SISTEM

3.1 Implementasi Database

Tahap implementasi diawali dengan pembuatan database sebagai media penyimpanan seluruh data pada Sistem Basis Data Restoran. Database yang digunakan bernama **restoran_db** dan dibuat menggunakan DBMS MySQL. Setelah database berhasil dibuat, database diaktifkan menggunakan perintah USE sehingga seluruh objek database seperti tabel, function, stored procedure, trigger, cursor, dan indexing disimpan pada database tersebut.

Implementasi SQL

```
CREATE DATABASE IF NOT EXISTS restoran_db;  
  
USE restoran_db;
```

Penjelasan:

Perintah CREATE DATABASE IF NOT EXISTS digunakan untuk membuat database apabila database tersebut belum tersedia pada server MySQL. Sedangkan perintah USE restoran_db digunakan untuk mengaktifkan database sehingga seluruh proses pembuatan objek database dilakukan pada database restoran_db.

3.2 Implementasi Tabel

Tahap berikutnya adalah pembuatan tabel yang digunakan untuk menyimpan seluruh data operasional restoran. Sistem ini terdiri dari enam tabel utama, yaitu tabel **pelanggan**, **kategori_menu**, **menu**, **pesanan**, **detail_pesanan**, dan **audit_log**.

Hubungan antar tabel dibangun menggunakan **Primary Key** dan **Foreign Key** sehingga integritas data tetap terjaga.

1. Implementasi Tabel Pelanggan

Tabel pelanggan digunakan untuk menyimpan identitas pelanggan yang melakukan transaksi pada restoran.

```
CREATE TABLE pelanggan (  
    id_pelanggan INT AUTO_INCREMENT PRIMARY KEY,  
    nama_pelanggan VARCHAR(100) NOT NULL,  
    nomor_meja INT NOT NULL,  
    no_telepon VARCHAR(15)  
);
```

Penjelasan:

Kolom id_pelanggan berfungsi sebagai Primary Key yang membedakan setiap pelanggan. Data pelanggan terdiri dari nama pelanggan, nomor meja, dan nomor telepon sehingga setiap transaksi dapat diketahui siapa pelanggannya.

2. Implementasi Tabel Kategori Menu

Tabel kategori_menu digunakan untuk mengelompokkan menu berdasarkan jenisnya.

```
CREATE TABLE kategori_menu (  
    id_kategori INT AUTO_INCREMENT PRIMARY KEY,  
    nama_kategori VARCHAR(50) NOT NULL  
);
```

Penjelasan:

Kategori menu digunakan agar proses pengelompokan makanan menjadi lebih terstruktur, misalnya makanan utama, minuman, camilan, dessert, dan paket hemat.

3. Implementasi Tabel Menu

Tabel menu digunakan untuk menyimpan informasi seluruh makanan dan minuman yang tersedia pada restoran.

```
CREATE TABLE menu (  
  
    id_menu INT AUTO_INCREMENT PRIMARY KEY,  
  
    id_kategori INT NOT NULL,  
  
    nama_menu VARCHAR(100) NOT NULL,  
  
    harga DECIMAL(10,2) NOT NULL,  
  
    stok INT NOT NULL DEFAULT 0,  
  
    FOREIGN KEY (id_kategori)  
    REFERENCES kategori_menu(id_kategori)  
  
    ON DELETE CASCADE  
  
);
```

Penjelasan:

Tabel menu memiliki relasi dengan tabel kategori_menu melalui atribut id_kategori. Selain nama menu dan harga, tabel ini juga menyimpan stok sehingga jumlah persediaan makanan dapat dipantau.

4. Implementasi Tabel Pesanan

Tabel pesanan digunakan untuk menyimpan transaksi yang dilakukan pelanggan.

```
CREATE TABLE pesanan (  
  
    id_pesanan INT AUTO_INCREMENT PRIMARY KEY,
```

```
id_pelanggan INT NOT NULL,  
  
tanggal_pesanan DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
total_bayar DECIMAL(10,2) DEFAULT 0.00,  
  
status_pembayaran ENUM('Belum Bayar','Lunas')  
  
DEFAULT 'Belum Bayar',  
  
FOREIGN KEY(id_pelanggan)  
  
REFERENCES pelanggan(id_pelanggan)  
  
);
```

Penjelasan:

Setiap transaksi memiliki satu pelanggan. Nilai total_bayar akan diperbarui secara otomatis setelah detail pesanan ditambahkan melalui stored procedure.

5. Implementasi Tabel Detail Pesanan

Tabel detail_pesanan menyimpan rincian menu yang dipesan pada setiap transaksi.

```
CREATE TABLE detail_pesanan (  
  
id_detail INT AUTO_INCREMENT PRIMARY KEY,  
  
id_pesanan INT NOT NULL,  
  
id_menu INT NOT NULL,  
  
jumlah INT NOT NULL,  
  
subtotal DECIMAL(10,2) NOT NULL,  
  
FOREIGN KEY(id_pesanan)  
  
REFERENCES pesanan(id_pesanan),  
  
FOREIGN KEY(id_menu)
```

```
REFERENCES menu(id_menu)

);
```

Penjelasan:

Satu transaksi dapat memiliki lebih dari satu menu. Oleh karena itu, tabel detail_pesanan digunakan sebagai tabel penghubung antara pesanan dan menu.

6. Implementasi Tabel Audit Log

Audit Log digunakan untuk mencatat aktivitas sistem yang dilakukan secara otomatis oleh trigger.

```
CREATE TABLE audit_log (

    id_log INT AUTO_INCREMENT PRIMARY KEY,

    aksi VARCHAR(50),

    tabel_terdampak VARCHAR(50),

    keterangan TEXT,

    waktu DATETIME DEFAULT CURRENT_TIMESTAMP

);
```

Penjelasan:

Data pada tabel ini tidak diinput secara manual. Seluruh data berasal dari trigger ketika terjadi perubahan status pesanan maupun penghapusan menu.

3.3 Implementasi Function

Function digunakan untuk melakukan proses perhitungan yang dapat dipanggil berulang kali oleh stored procedure.

1. Function fn_hitung_subtotal

Function ini digunakan untuk menghitung subtotal berdasarkan harga menu dan jumlah menu yang dipesan.

Penjelasan:

Function mengambil harga dari tabel menu berdasarkan id_menu, kemudian mengalikan harga tersebut dengan jumlah pesanan sehingga menghasilkan nilai subtotal.

2. Function fn_hitung_diskon

Function ini digunakan untuk menghitung potongan harga berdasarkan total pembayaran.

Aturan diskon

- Total $\geq$ Rp100.000 $\rightarrow$ Diskon 10%
- Total $\geq$ Rp50.000 $\rightarrow$ Diskon 5%
- Selain itu $\rightarrow$ Tidak mendapat diskon

Function ini digunakan pada laporan penjualan harian maupun rekap bulanan.

3.4 Implementasi Stored Procedure

Stored Procedure digunakan untuk mengelompokkan beberapa perintah SQL menjadi satu prosedur sehingga proses transaksi menjadi lebih mudah dan lebih efisien.

Pada sistem ini terdapat enam Stored Procedure.

1. sp_tambah_pesanan

Digunakan untuk membuat transaksi pesanan baru.

2. sp_tambah_detail_pesanan

Digunakan untuk menambahkan menu ke dalam pesanan, menghitung subtotal, memeriksa stok, dan memperbarui total pembayaran.

3. sp_laporan_penjualan_harian

Digunakan untuk menampilkan laporan transaksi berdasarkan tanggal tertentu.

4. sp_proses_rekap_bulanan

Digunakan untuk menghasilkan laporan penjualan bulanan menggunakan Cursor.

5. sp_tambah_pelanggan

Digunakan untuk menambahkan data pelanggan baru ke dalam tabel pelanggan.

6. sp_tambah_menu

Digunakan untuk menambahkan menu baru ke dalam tabel menu.

3.5 Implementasi Trigger

Sistem menggunakan tiga trigger.

1. Trigger trg_kurangi_stok

Trigger ini akan berjalan setiap kali data baru ditambahkan ke tabel detail_pesanan. Fungsi utamanya adalah mengurangi stok menu secara otomatis sesuai jumlah yang dipesan.

2. Trigger trg_audit_pesanan_selesai

Trigger ini akan mencatat setiap perubahan status pembayaran dari **Belum Bayar** menjadi **Lunas** ke dalam tabel audit_log.

3. Trigger trg_audit_hapus_menu

Trigger ini akan mencatat informasi menu yang dihapus ke dalam tabel audit_log sebagai bentuk pencatatan aktivitas sistem.

3.6 Implementasi Cursor

Cursor digunakan pada prosedur sp_proses_rekap_bulanan untuk membaca data transaksi yang telah berstatus **Lunas** satu per satu. Setiap transaksi diproses dengan menghitung diskon menggunakan function, kemudian hasilnya dimasukkan ke dalam tabel sementara sebelum ditampilkan sebagai laporan rekap penjualan bulanan.

3.7 Implementasi Transaction Control

Transaction Control diterapkan untuk menjaga konsistensi data selama proses transaksi berlangsung. Sistem menggunakan perintah:

- START TRANSACTION
- SAVEPOINT
- COMMIT
- ROLLBACK

Dengan mekanisme ini, setiap perubahan data dapat disimpan apabila berhasil atau dibatalkan apabila terjadi kesalahan sehingga integritas database tetap terjaga.

3.8 Implementasi Exception Handling

Exception Handling diterapkan pada prosedur `sp_tambah_detail_pesanan`. Ketika ID menu atau ID pesanan tidak ditemukan, sistem akan menghasilkan pesan kesalahan. Selain itu, apabila stok menu lebih kecil daripada jumlah yang dipesan, sistem juga akan menghentikan proses transaksi dan menampilkan informasi bahwa stok tidak mencukupi. Dengan demikian, data yang tersimpan tetap valid dan konsisten.

3.9 Implementasi Indexing

Untuk meningkatkan kecepatan pencarian data, sistem menggunakan dua indeks, yaitu:

- **idx_nama_menu** pada kolom `nama_menu` di tabel `menu`.
- **idx_tanggal_pesanan** pada kolom `tanggal_pesanan` di tabel `pesanan`.

Penggunaan indeks ini membuat proses pencarian menu dan penyusunan laporan berdasarkan tanggal menjadi lebih cepat, terutama ketika jumlah data dalam database semakin besar.

BAB IV

PENGUJIAN

4.1 Skenario Pengujian

Tahap pengujian dilakukan untuk memastikan seluruh objek database yang telah dibuat dapat berfungsi sesuai dengan tujuan perancangan. Pengujian meliputi tabel, stored procedure, function, trigger, cursor, transaction control, exception handling, dan indexing. Seluruh pengujian dilakukan menggunakan phpMyAdmin pada DBMS MySQL.

Berikut skenario pengujian yang dilakukan.

No	Objek yang Diuji	Tujuan Pengujian
1	Tabel	Memastikan seluruh tabel berhasil dibuat
2	Insert Data	Memastikan data dummy berhasil disimpan
3	Function	Menghitung subtotal dan diskon
4	Stored Procedure	Menambah pesanan dan menampilkan laporan
5	Trigger	Mengurangi stok dan mencatat audit log
6	Cursor	Rekap penjualan bulanan
7	Transaction Control	Commit dan Rollback
8	Exception Handling	Menampilkan pesan kesalahan
9	Indexing	Mempercepat pencarian data

4.2 Hasil Eksekusi Query

Pengujian pertama dilakukan untuk memastikan seluruh tabel berhasil dibuat dan data awal berhasil dimasukkan ke dalam database.

1. Pengujian Data Pelanggan

SELECT * FROM pelanggan;

Hasil yang diharapkan

Data seluruh pelanggan tampil.

Screenshot

The screenshot shows the phpMyAdmin interface with the 'pelanggan' table selected in the 'restoran_db' database. The table contains 6 rows of customer data. The SQL query 'SELECT * FROM pelanggan;' is entered in the query box. The table structure is as follows:

id_pelanggan	nama_pelanggan	nomor_meja	no_telepon
1	Budi Santoso	1	081234567890
2	Siti Aminah	2	082198765432
3	Andi Pratama	3	083811223344
4	Dewi Lestari	4	085755667788
5	Rian Hidayat	5	089699001122
6	Bambang Permadi	6	081344332211

2. Pengujian Data Menu

SELECT * FROM menu;

Hasil yang diharapkan

Daftar menu beserta harga dan stok berhasil ditampilkan.

Screenshot

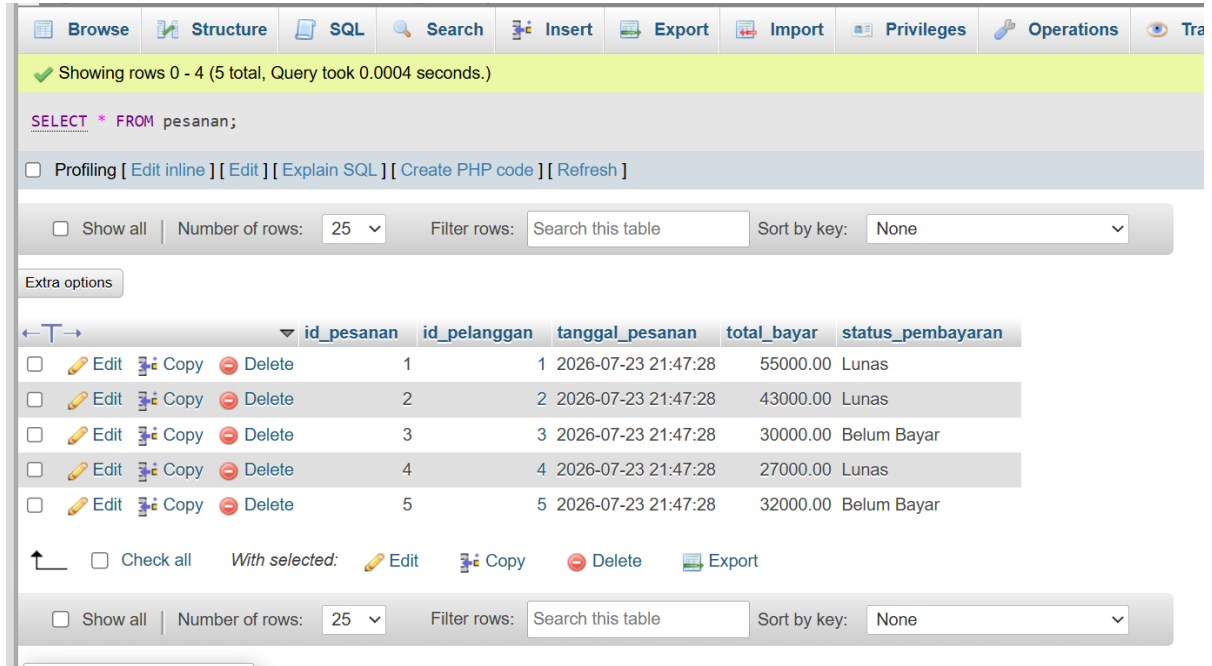
The screenshot shows the phpMyAdmin interface with the 'menu' table selected in the 'restoran_db' database. The table contains 10 rows of menu data. The SQL query 'SELECT * FROM menu;' is entered in the query box. The table structure is as follows:

id_menu	id_kategori	nama_menu	harga	stok
1	1	Nasi Goreng Spesial	25000.00	50
2	1	Ayam Bakar Madu	30000.00	40
3	1	Mie Goreng Seafood	28000.00	35
4	1	Soto Ayam Kampung	22000.00	30
5	2	Es Teh Manis	5000.00	100
6	2	Jus Alpukat	15000.00	40
7	2	Kopi Hitam	10000.00	60
8	3	Kentang Goreng	15000.00	45
9	4	Es Krim Vanilla	12000.00	25
10	5	Paket Nasi + Ayam + Es Teh	32000.00	50

3. Pengujian Data Pesanan

```
SELECT * FROM pesanan;
```

Hasil yang di harapkan



Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.)

`SELECT * FROM pesanan;`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				id_pesanan	id_pelanggan	tanggal_pesanan	total_bayar	status_pembayaran
☐	Edit	Copy	Delete	1	1	2026-07-23 21:47:28	55000.00	Lunas
☐	Edit	Copy	Delete	2	2	2026-07-23 21:47:28	43000.00	Lunas
☐	Edit	Copy	Delete	3	3	2026-07-23 21:47:28	30000.00	Belum Bayar
☐	Edit	Copy	Delete	4	4	2026-07-23 21:47:28	27000.00	Lunas
☐	Edit	Copy	Delete	5	5	2026-07-23 21:47:28	32000.00	Belum Bayar

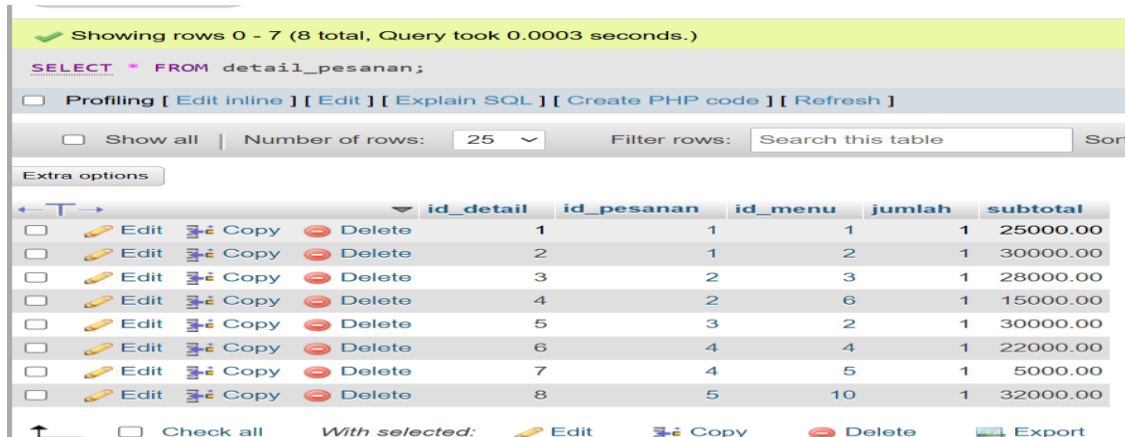
☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

4. Pengujian Detail Pesanan

```
SELECT * FROM detail_pesanan;
```

Hasil yang di harapkan



Showing rows 0 - 7 (8 total, Query took 0.0003 seconds.)

`SELECT * FROM detail_pesanan;`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort

Extra options

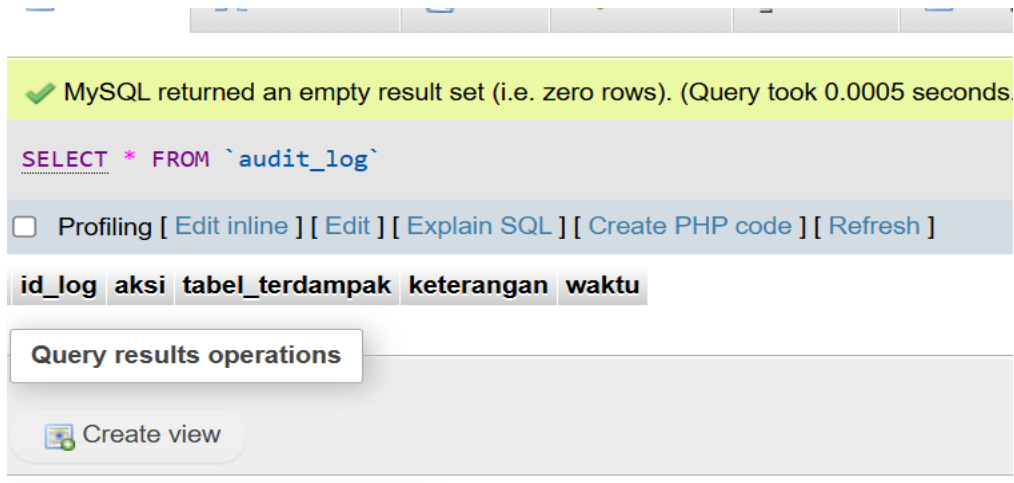
				id_detail	id_pesanan	id_menu	jumlah	subtotal
☐	Edit	Copy	Delete	1	1	1	1	25000.00
☐	Edit	Copy	Delete	2	1	2	1	30000.00
☐	Edit	Copy	Delete	3	2	3	1	28000.00
☐	Edit	Copy	Delete	4	2	6	1	15000.00
☐	Edit	Copy	Delete	5	3	2	1	30000.00
☐	Edit	Copy	Delete	6	4	4	1	22000.00
☐	Edit	Copy	Delete	7	4	5	1	5000.00
☐	Edit	Copy	Delete	8	5	10	1	32000.00

☐ Check all | With selected: Edit Copy Delete Export

5. Pengujian Audit Log

```
SELECT * FROM audit_log;
```

Hasil yang di harapkan



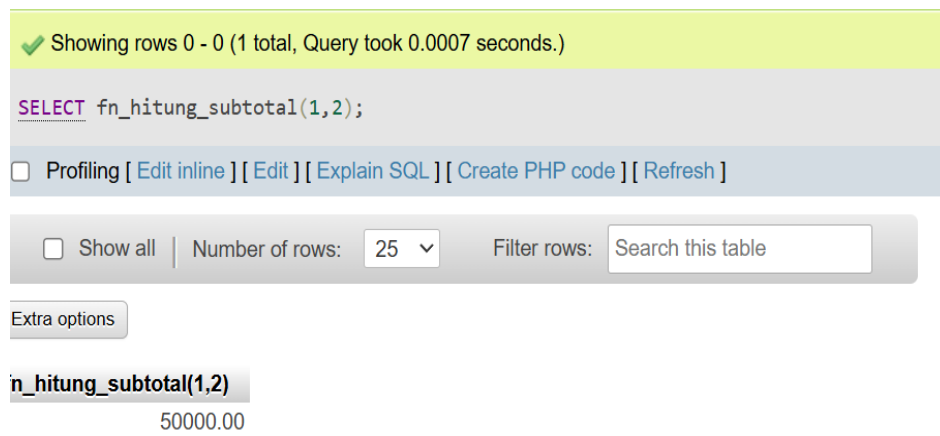
4.3 Pengujian Function

Function Menghitung Subtotal

```
SELECT fn_hitung_subtotal(1,2);
```

Hasil yang diharapkan

Harga menu ID 1 dikalikan jumlah 2 menghasilkan subtotal sesuai data menu.

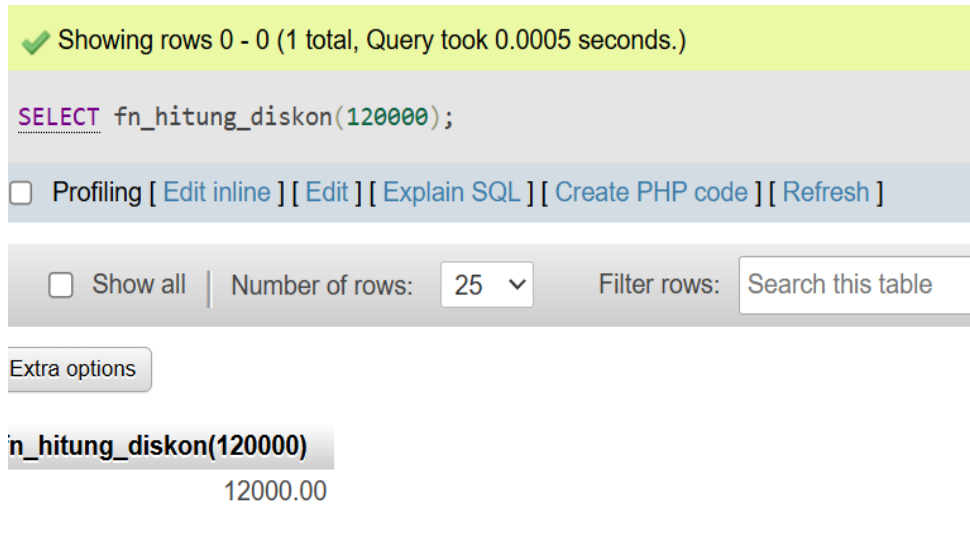


Function Menghitung Diskon

```
SELECT fn_hitung_diskon(120000);
```

Hasil yang diharapkan

Diskon sebesar 10%.



Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)

```
SELECT fn_hitung_diskon(120000);
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

fn_hitung_diskon(120000)
12000.00

4.4 Pengujian Stored Procedure

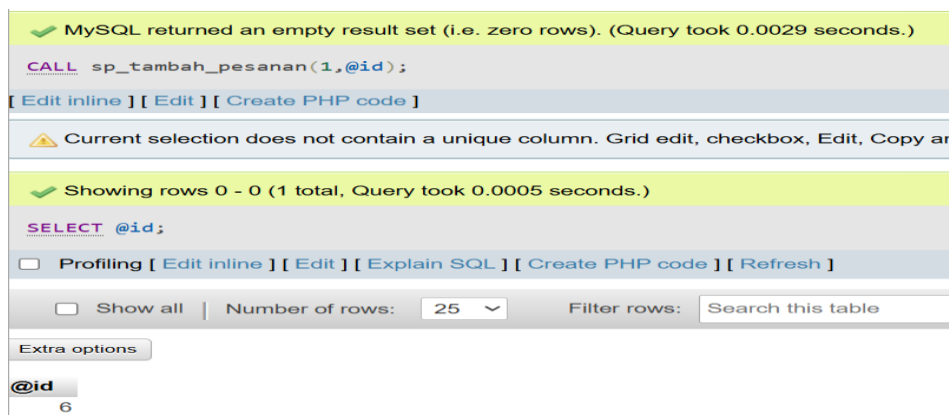
1. Menambah Pesanan

```
CALL sp_tambah_pesanan(1,@id);
```

```
SELECT @id;
```

Hasil yang diharapkan

Pesanan baru berhasil dibuat dan menghasilkan ID pesanan baru.



MySQL returned an empty result set (i.e. zero rows). (Query took 0.0029 seconds.)

```
CALL sp_tambah_pesanan(1,@id);
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy ar

Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)

```
SELECT @id;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

@id
6

2. Menambah Detail Pesanan

CALL sp_tambah_detail_pesanan(1,1,2);

Hasil yang diharapkan

Detail pesanan berhasil ditambahkan dan total pembayaran bertambah.

Query editor

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0046 seconds.)

CALL sp_tambah_detail_pesanan(1,1,2);

Edit inline ][ Edit ][ Create PHP code ]
```

3. Menampilkan Laporan Harian

CALL sp_laporan_penjualan_harian(CURDATE());

Hasil yang diharapkan

Data transaksi pada tanggal tersebut berhasil ditampilkan lengkap dengan diskon.

CALL sp_laporan_penjualan_harian(CURDATE());

Edit inline][Edit][Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

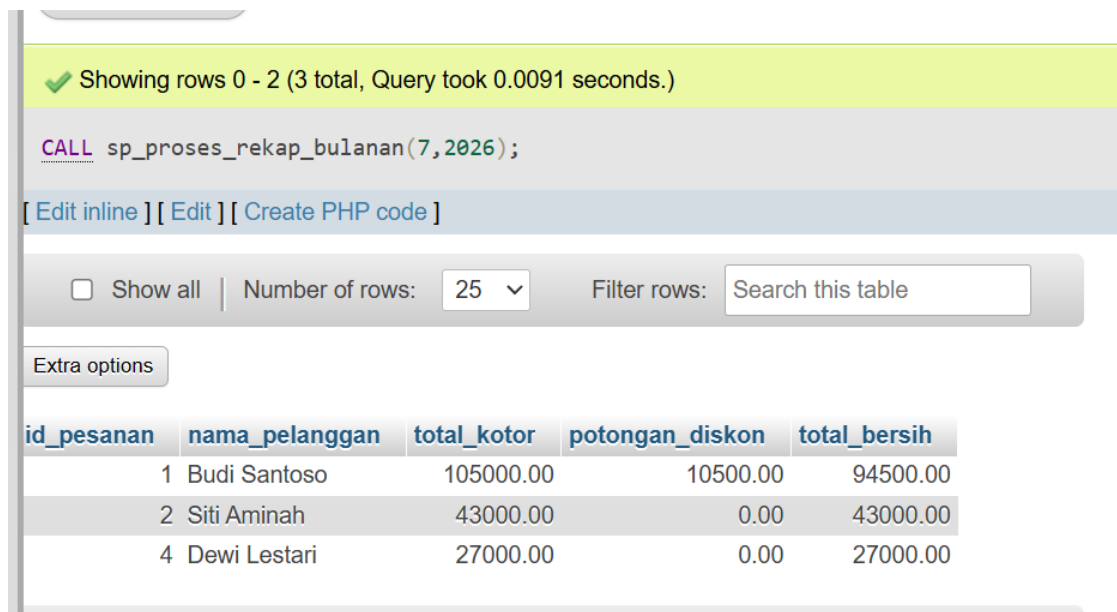
i_pesanan	nama_pelanggan	nomor_meja	tanggal_pesanan	total_bayar	diskon	total_setelah_diskon	status_pembayaran
1	Budi Santoso	1	2026-07-23 21:47:28	105000.00	10500.00	94500.00	Lunas
2	Siti Aminah	2	2026-07-23 21:47:28	43000.00	0.00	43000.00	Lunas
3	Andi Pratama	3	2026-07-23 21:47:28	30000.00	0.00	30000.00	Belum Bayar
4	Dewi Lestari	4	2026-07-23 21:47:28	27000.00	0.00	27000.00	Lunas
5	Rian Hidayat	5	2026-07-23 21:47:28	32000.00	0.00	32000.00	Belum Bayar
6	Budi Santoso	1	2026-07-23 22:10:38	0.00	0.00	0.00	Belum Bayar

4. Rekap Bulanan

CALL sp_proses_rekap_bulanan(7,2026);

Hasil yang diharapkan

Rekap transaksi bulan Juli 2026 berhasil ditampilkan.



Showing rows 0 - 2 (3 total, Query took 0.0091 seconds.)

CALL sp_proses_rekap_bulanan(7,2026);

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

id_pesanan	nama_pelanggan	total_kotor	potongan_diskon	total_bersih
1	Budi Santoso	105000.00	10500.00	94500.00
2	Siti Aminah	43000.00	0.00	43000.00
4	Dewi Lestari	27000.00	0.00	27000.00

4.5 Bukti Trigger Berjalan

Trigger Pengurangan Stok

Sebelum menjalankan prosedur penambahan detail pesanan, dilakukan pengecekan stok menu.

```
SELECT stok  
  
FROM menu  
  
WHERE id_menu=1;
```

✓ Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT stok FROM menu WHERE id_menu=1;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

		stok
☐	Edit Copy Delete	48

Kemudian jalankan.

```
CALL sp_tambah_detail_pesanan(1,1,2);
```

Setelah itu.

```
SELECT stok  
  
FROM menu  
  
WHERE id_menu=1;
```

Hasil

Stok menu berkurang sebanyak dua.

✓ Showing rows 0 - 0 (1 total, Query took 0.0016 seconds.)

```
SELECT stok FROM menu WHERE id_menu=1;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

		stok
☐	Edit Copy Delete	46

4.6 Trigger Audit Status Pesanan

```
UPDATE pesanan
SET status_pembayaran='Lunas'
WHERE id_pesanan=3;
```

✓ 1 row affected. (Query took 0.0004 seconds.)

```
UPDATE pesanan SET status_pembayaran='Lunas' WHERE id_pesanan=3;
```

[Edit inline] [Edit] [Create PHP code]

Kemudian.

```
SELECT *
FROM audit_log;
```

Hasil

Muncul data baru pada audit log mengenai perubahan status pembayaran.

✓ Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT * FROM audit_log;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

	id_log	aksi	tabel_terdampak	keterangan	waktu
☐ Edit Copy Delete	1	UPDATE STATUS	pesanan	Pesanan ID 3 mengubah status dari Belum Bayar menjadi Lunas	2026-07-23 22:19:21

4.7 Trigger Audit Hapus Menu

```
DELETE FROM menu
WHERE id_menu=10;
```

✓ 1 row deleted. (Query took 0.0032 seconds.)

```
DELETE FROM menu WHERE id_menu=10;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

Kemudian.

```
SELECT *
FROM audit_log;
```

Hasil

Audit log mencatat informasi menu yang dihapus.

✓ Showing rows 0 - 1 (2 total, Query took 0.0003 seconds.)

```
SELECT * FROM audit_log;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: | Filter rows: | Sort by key:

Extra options

				id_log	aksi	tabel_terdampak	keterangan	waktu
☐				1	UPDATE STATUS	pesanan	Pesanan ID 3 mengubah status dari Belum Bayar menjadi Lunas	2026-07-23 22:19:21
☐				2	DELETE	menu	Menu dihapus: Paket Nasi + Ayam + Es Teh (ID: 10)	2026-07-23 22:21:09

4.8 Bukti Transaction Control

```
COMMIT
```

```
START TRANSACTION;
```

```
INSERT INTO pelanggan
```

```
(nama_pelanggan,nomor_meja,no_telepon)
```

```
VALUES
```

```
('Data Commit',21,'081122334455');
```

```
COMMIT;
```

Hasil

Data tetap tersimpan.

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
START TRANSACTION;
```

[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ 1 row inserted.

Inserted row id: 7 (Query took 0.0004 seconds.)

```
INSERT INTO pelanggan (nama_pelanggan,nomor_meja,no_telepon) VALUES ('Data Commit',21,'081122334455');
```

[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0022 seconds.)

```
COMMIT;
```

[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

ROLLBACK

```
START TRANSACTION;
```

```
INSERT INTO pelanggan
```

```
(nama_pelanggan,nomor_meja,no_telepon)
```

```
VALUES

('Data Rollback',22,'08111111111');

ROLLBACK;
```

Hasil

Data tidak tersimpan.

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0001 seconds.)

```
START TRANSACTION;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ 1 row inserted.

Inserted row id: 8 (Query took 0.0004 seconds.)

```
INSERT INTO pelanggan (nama_pelanggan,nomor_meja,no_telepon) VALUES ('Data Rollback',22,'08111111111');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0020 seconds.)

```
ROLLBACK;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

4.9 Bukti Audit Log

Audit log digunakan untuk mencatat aktivitas penting yang dilakukan oleh trigger.

```
SELECT *

FROM audit_log;

Contoh isi audit log:
```

ID	Aksi	Tabel
1	UPDATE STATUS	pesanan
2	DELETE	menu

Screenshot

Showing rows 0 - 1 (2 total, Query took 0.0003 seconds.)

SELECT * FROM audit_log;

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

				id_log	aksi	tabel_terdampak	keterangan	waktu
☐	Edit	Copy	Delete	1	UPDATE STATUS	pesanan	Pesanan ID 3 mengubah status dari Belum Bayar menj...	2026-07-23 22:19:21
☐	Edit	Copy	Delete	2	DELETE	menu	Menu dihapus: Paket Nasi + Ayam + Es Teh (ID: 10)	2026-07-23 22:21:09

4.10 Perbandingan Query Sebelum dan Sesudah Indexing

Sebelum Index

```
SELECT *  
  
FROM menu  
  
WHERE nama_menu='Nasi Goreng Spesial';
```

Proses pencarian dilakukan dengan membaca seluruh data pada tabel

Sesudah Index

```
CREATE INDEX id_nama_menu  
  
ON menu(nama_menu);  
  
Kemudian jalankan kembali.  
  
SELECT *  
  
FROM menu  
  
WHERE nama_menu='Nasi Goreng Spesial';
```


Hasil

Proses pencarian menjadi lebih cepat karena MySQL menggunakan indeks pada kolom nama_menu.

 Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

```
SELECT * FROM menu WHERE nama_menu='Nasi Goreng Spesial';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: Filter rows:

Extra options

			id_menu	id_kategori	nama_menu	harga	stok	
☐	 Edit	 Copy	 Delete	1	1	Nasi Goreng Spesial	25000.00	46

BAB V

KESIMPULAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian, Sistem Basis Data Restoran berhasil dibangun menggunakan MySQL sesuai dengan kebutuhan proyek Evaluasi Akhir Semester. Sistem terdiri dari enam tabel utama yang saling berelasi menggunakan primary key dan foreign key sehingga integritas data dapat terjaga. Selain itu, sistem juga berhasil mengimplementasikan berbagai fitur lanjutan MySQL, yaitu Stored Procedure, Function, Trigger, Cursor, Transaction Control, Exception Handling, dan Indexing.

Hasil pengujian menunjukkan bahwa seluruh objek database dapat berjalan sesuai dengan fungsinya. Stored Procedure mampu mengelola transaksi dan menghasilkan laporan, Function berhasil melakukan perhitungan subtotal serta diskon, Trigger mampu mengurangi stok dan mencatat aktivitas ke audit log secara otomatis, Cursor berhasil menyusun rekap penjualan bulanan, Transaction Control menjaga konsistensi data, Exception Handling mampu menangani kesalahan input, dan Indexing meningkatkan efisiensi proses pencarian data.

Dengan demikian, sistem basis data restoran yang dibangun telah memenuhi tujuan proyek dan dapat digunakan sebagai dasar pengembangan sistem informasi restoran yang lebih lengkap.

5.2 Kendala

Selama proses pengembangan proyek terdapat beberapa kendala yang dihadapi, antara lain:

1. Menentukan relasi antar tabel agar tidak terjadi redundansi data.
2. Menyesuaikan sintaks MySQL untuk pembuatan Stored Procedure, Function, Trigger, dan Cursor.
3. Mengatasi kesalahan (error) pada penggunaan delimiter dan foreign key.
4. Memastikan setiap trigger dan transaction berjalan sesuai skenario pengujian.
5. Melakukan pengujian agar seluruh objek database saling terintegrasi dengan baik.

5.3 Saran Pengembangan

Beberapa saran pengembangan untuk sistem ini adalah:

1. Menambahkan antarmuka berbasis web atau desktop agar lebih mudah digunakan.
2. Menambahkan fitur autentikasi pengguna sehingga setiap pengguna memiliki hak akses yang berbeda.
3. Mengembangkan sistem agar dapat mencetak nota pembayaran secara otomatis.
4. Menambahkan laporan penjualan berdasarkan rentang tanggal, kategori menu, dan pelanggan.
5. Mengintegrasikan sistem dengan aplikasi kasir (Point of Sale) sehingga proses transaksi dapat dilakukan secara real-time.